A real `window` object in a web browser has more than 200 properties. It would take a very long time to create a suitable double to use inside the test. Instead of writing a double from scratch, you can use `Object.create` to create a double with the real thing as a prototype and then override just the parts you need to. For the `Form` class you need to override the `reload` method:

```
windowDouble = Object.create window
windowDouble.location.reload = ->
```

This saves some time in creating a double to inject during testing. Change the `Form` class to accept an injected window in the constructor:

```
class Form
  constructor: (@window) ->
  reloader: ->
    @window.location.reload()
```

Isolation

The `extractData` function does too much. It fetches and extracts the data when really it should just extract the data. Breaking out the fetching of data into a separate function removes the dependency that `extractData` has on `http`. This means that tests for `extractData` don't need to even know about `http`, let alone create a double:

```
fetchData (http, callback) ->
  options =
    host: 'www.agtronscameras.com'
    port: 80
    path: "/data/#{topic}"

  http.get(options, (res) ->
    callback res
  ).on 'error', (e) ->
    console.log e

extractData = (data) ->
  console.log data

fetchAndExtractData = ->
  fetchData http, extractData
```